

Multi-Agent Architecture

<https://github.com/aiyeshwary/multi-agent-revenue-churn>

OVERVIEW

A production-grade agentic pipeline that combines LLM-driven planning with deterministic Python analytics for churn prediction, featuring an Orchestrator-Executor-Reflection loop with memory management and Docker.

IMPACT

Designed and implemented a multi-agent revenue and churn analysis system where an LLM Planner coordinates a Reflection agent, keeping LLMs focused on reasoning while Python handles all numeric computation.

KEY DETAILS

1. Planner emits an executable plan (list or graph of nodes) from a high-level task description.
2. Orchestrator runs each step via the Executor, applies deterministic validators, and uses Reflection for failed actions.
3. Memory Manager persists short summaries and semantic snippets for context retrieval across sessions.
4. Outputs churn_report.json, per-client quarterly revenue CSVs, and HTML/PNG visualizations per client.
5. Fully containerized with Docker Compose; CI/CD via GitHub Actions with pytest unit test suite.
6. GraphRunner resolves step dependencies as a DAG and executes ready nodes in parallel via asyncio with CircuitBreaker and SHA-256 idempotency keys to prevent duplicate execution.
7. SemanticMemory uses a dependency-free in-memory cosine-similarity index with deterministic embeddings and fully reproducible tests; long-term memory uses zlib compression and persists to JSON.
8. LLMClient supports both a mock provider (deterministic stubs for offline testing) and OpenAI provider. TokenEstimator enforces budgets before every call. Visualizations generated with Matplotlib (HTML).